GHCAA Software Requirements Specification

Version 2.6. Printed 2026-10-31.

Contents at a glance

Section	Items
Membership & Identity Lifecycle	6
Event Management & Participation	4
Financial Governance & Payments	4
Networking & Social	4
In-App Assistant (Haraganga Assistant)	2
Governance & CMS	12
Alumni Programs & Verification	3
Security & Data Integrity	6
Performance & UI/UX	3
Total	**44**

1. Project Introduction

1.1 Purpose

The GHCAA (Govt. Haraganga College Alumni Association) Platform is the system of record for the alumni network: membership management, financial transparency, event coordination, and professional networking.

1.2 Scope

The system consists of a **REST API (ASP.NET Core)**, a **Web Portal (Angular)**, and a **Mobile Application (Flutter)**, covering the full member lifecycle from registration through ongoing engagement.

1.3 Vision

Give alumni secure, real-time tools to stay in touch with each other and to support the college.

2. System Architecture & Tech Stack

The platform follows **Clean Architecture** principles so business logic stays independent of frameworks and databases.

Layer	Technology	Role
Backend	ASP.NET Core 9.0 (C#)	RESTful API, RBAC, SignalR Hubs
Database	PostgreSQL 16+	Relational storage with Global Query Filters
Frontend	Angular 21 (Signals)	Admin Command Center & Public Portal
Mobile	Flutter (Dart)	Member Hub (iOS/Android)
Assistant	Rule-based engine (C#)	Intent/keyword classifier over internal data
Real-time	SignalR	Instant Notifications & Peer Messaging
Auth	JWT + BCrypt	Stateless Authentication & Secure Hashing

3. Functional Requirements (FR)

3.1 Membership & Identity Lifecycle

The core of the system is a high-integrity registration and verification workflow.

- **3.1.1 Registration Wizard**: A guided multi-step process (Personal, Academic, Professional, Attachments, Verification).
- *Constraint*: Registration requires at least one record from "Govt. Haraganga College".
- **3.1.2 Approval Workflow**: Admins review "Applied" users. Upon approval:
- A unique **Membership Number** is generated (e.g., `GHC-[Year]-[Serial]`).
- System auto-provisions a **User Account** where the Membership Number becomes the Login ID.
- **3.1.3 Digital ID & Certificates**: Automatic generation of secure, QR-coded SVG/PDF ID cards and membership certificates for "Active" members.
- **3.1.4 Profile Management**: Granular control over personal data with member-defined **Privacy Toggles** (Masking NID, Email, Mobile, Address).
- **3.1.5 Social Authentication**: Integration with **Google** and **Facebook** for simplified login. The system automatically links social IDs to existing accounts via email or prompts for profile completion for new users. A provider's button is offered only when `Features.EnableSocialAuth` is on, the provider is enabled, and its client credentials are set.
- **3.1.6 Onboarding Workflow**: "Applied" members who login via social auth are restricted to a Profile Wizard. Full system access is gated until:
- **Profile Completion**: 100% of mandatory fields provided. The 13 required fields are:
  - Personal: `FullName`, `Email`, `MobileNo`, `DateOfBirth`, `Gender`, `NID`, `FatherName`, `MotherName`, `PermanentAddress`, `BloodGroup`.
  - Visual: `PhotoPath`.
  - Academic: At least one record from "Govt. Haraganga College" (`AcademicHistory`).
  - Professional: At least one professional experience record (`ProfessionalHistory`).
- **Financial Commitment**: Successful payment of registration/membership fees.
- **Admin Approval**: Formal verification by the association secretariat.

3.2 Event Management & Participation

A central hub for organizing physical and virtual gatherings.

- **3.2.1 Event Catalog**: Discovery hub with capacity capping, deadlines, and guest entry policies.
- **3.2.2 Registration & Waitlisting**: Automated handling of participant limits and waitlist transitions.
- **3.2.3 QR Attendance Tracking**: Coordinators use the Mobile App to scan Member ID QR codes for real-time attendance logging.
- **3.2.4 Administrative Controls**: Management of event tasks, budgets, expenses, and gallery documentation.

3.3 Financial Governance & Payments

Transparent, automated revenue collection.

- **3.3.1 Immutable Ledger**: Tracking all income (Dues, Donations, Fees) and expenses with audit-ready records.
- **3.3.2 Automated Dues**: Generation and notification of annual membership fees based on configurable types (General, Life, etc.).
- **3.3.3 Manual, Admin-Configurable Payments**: The platform operates without live payment-gateway credentials. Administrators configure wallet, bank, and mobile-financial-service (**bKash**, **Nagad**, bank transfer) instructions; members pay through the displayed channel and upload proof of payment, which an administrator verifies and approves. Automated gateway callback handling (**SSLCommerz**, **bKash**, **Nagad** webhooks) exists in the codebase and can be enabled later by supplying gateway keys, but is not required for operation.
- **3.3.4 Digital Receipts**: Instant generation of PDF tax receipts for all successful contributions.

3.4 Networking & Social

Supporting professional and personal connections within the network.

- **3.4.1 Alumni Directory**: "Infinite Scroll" directory with batch, department, and professional filtering.
- **3.4.2 Peer-to-Peer Messaging**: Real-time SignalR-powered chat between members without exposing private contact info.
- **3.4.3 Job & Mentorship Hub**: Internal boards for sharing career opportunities and requesting formal mentorship.
- **3.4.4 Gamification**: "Contribution Points" and "Badges" (Legend, Elite, Active) awarded for engagement and attendance.

3.5 In-App Assistant (Haraganga Assistant)

- **3.5.1 Guided Search**: A rule-based intent/keyword classifier that maps member questions to internal lookups (e.g., finding alumni by batch or profession) over existing directory data — no external LLM dependency.
- **3.5.2 Policy Support**: Answers association rules and portal-navigation questions from a curated internal knowledge base. (The service is structured so a generative provider could be added later without changing the API surface.)

3.6 Governance & CMS

- **3.6.1 EC Management**: Management of Executive Committee terms, roles (President, GS, etc.), and historical governance records.
- **3.6.2 Constitution Hub**: Version-controlled governing documents with member voting capabilities. **The public reader always serves the latest ratified version**: it renders whichever version row is active, rebuilds its article navigation from that text, and lists superseded versions in a history panel with their own PDF links. No page, link or component may pin a version number, so ratifying a new document changes what every surface shows without a code change to those surfaces. The currently published text is v4.2, effective 01 Jul 2026. Section labels within an article are rendered as subheadings so a long article is navigable rather than an undifferentiated wall of text.
- **3.6.2b Public Election Board and Forms From Records**: `GET api/elections/{id}/board` is anonymous and lists the accepted, signed officials who chose to appear, grouped by role, with no contact fields. Turnout shows from Counting on, or during Polling when `ShowTurnoutDuringPolling` is on, and only once at least `MinimumBallotsToCount` eligible voters have voted. The audit head hash shows from Declared on. A test election answers with `isTest` set. The server-rendered forms ER-09, 10, 13, 14, 19, 23, 29, 30 and 31 take their tables from stored records, so ER-23 prints the counts of the run.
- **3.6.2a Election Document Library & Form Pad**: The seven election procedure documents are published publicly, with the forms handbook split into 18 individually viewable, downloadable and printable forms. Each form renders as an **operative A4 document, not a specimen** — association letterhead, reference and date line, form code, ruled write-on fields, tick boxes, signature grid and seal box — so it can be printed, filled in by hand and signed. Every organisational string in the letterhead (name, crest, address, phone, email, motto, founding date) is drawn from organisation configuration, so a re-branded deployment prints its own forms without a code change. The sheet is ink-on-white regardless of the reader's chosen theme, because it is a paper document.
- **3.6.3 Communication Hub**: Mass email/SMS engine with HTML templates and targeted segmenting (e.g., by batch or type).
- **3.6.4 Dynamic Themes**: UI transformations for special days (e.g., National Days) controlled via Admin settings.
- **3.6.5 Site Content Blocks**: Admin-editable, keyed content blocks (title, rich-text body, display order, active flag) that drive the public About Us page and the Contact page intro, so institutional copy is changed from the admin panel rather than by redeploy. Block bodies are sanitized server-side on every write; the public page retains a static fallback so it can never render empty.
- **3.6.6 News & Notice Board**: News and official notices share one entity discriminated by `PostType`, managed from a single admin screen and surfaced on one tab-filtered public/portal feed. Notices may carry a PDF attachment. **Notices may be posted by administrators only** — the member article-submission path rejects notice-typed posts from non-admin callers, while member *news* submissions continue to follow the existing approval workflow.
- **3.6.7 Contact Details Configuration**: On-campus address, phone numbers, support email, social links, and an optional map embed URL are held in organisation configuration and edited by SuperAdmin, giving web and mobile a single source of truth. The map URL is admin-supplied and therefore allow-list validated before being embedded.

- **3.6.8 Member Polls & Voting**: Admin-managed polling system for sentiment analysis and formal association decisions. Supports single/multiple choice, expiry dates, and real-time result visualization for members.
- **3.6.9 Election Officials, Rules Freeze and Rules Unlock**: Administrators appoint and revoke election officials from an officials panel on the admin elections page (web) and an officials sheet (mobile). The API enforces these rules, and a screen that hides a button is not enough:
- *Freeze*: while any election has an open-polling request waiting for approval, or is in Polling or Counting, the shared election settings, ballot and count rules, personas and permissions, appointments and the ballot key cannot change. A refused change answers 409 and is audited.
- *Rules unlock*: a SuperAdmin with step-up can open a time-boxed unlock (30 minutes by default, 60 at most) with a written reason of 20 to 1000 characters. Every change it lets through is audited with the unlock id and reason. It never covers a plain appointment revoke.
- *Emergency revoke*: while the rules are frozen an official is removed by an emergency revoke. It needs another active official on that election who holds Approve and who is neither the requester nor the target. SuperAdmin has no bypass. A request nobody acts on expires, and a background sweep every 15 minutes writes the `expired` audit row.
- *Returning Officer*: identified by a flag on the appointment, one live Returning Officer per election.
- *Access ends by itself*: declaring the result sets an end date on each appointment, the declared date plus the `AccessEndsDaysAfterDeclare` setting (default 21 days). After it the appointment is no longer live, with no scheduled job. A non-member official whose only role is ElectionOfficial is refused at login, social login and token refresh, and the account is deactivated. A member official keeps member access. Archiving the election revokes every appointment, removes the role, rotates the security stamp and revokes refresh tokens.
- *Election audit chain*: each election keeps a hash-chained list of its own events. Every entry carries the hash of the one before it, so a changed, removed or reordered entry is found by `GET api/elections/{id}/audit/verify`, which anyone can call and which names the first broken entry. The list itself is for officials who hold ViewAudit. Entries hold no voter or ballot identity.
- *Opened ballots*: once the result is declared, and when the `PublishPerSeatBallots` setting is on, `GET api/elections/{id}/opened-ballots` lists the opened ballots per seat as nomination ids. The order within a seat is fixed between reads and unrelated between seats, so a ballot cannot be followed across seats.
- *Vote audit device details*: the audit row for a cast ballot holds the device class, OS and browser parsed from the user agent, and the date without a time of day.
- *Count*: approving a count does not run it. The requester runs it with the key file. Unless the `CountRequesterOnly` org setting (default off) is on, any official with Count other than the approver may run it. The executor is recorded.
- *Readiness gate*: polling opens only when the voter roll is frozen, every accepted candidate has a ballot place, the ballot key is set, every open appointment is accepted with its declaration signed, the polling window is valid, and officials with ChangePhase have recorded the mock election, technical audit, decided appeals and paper stock. A request to open polling and its approval answer 400 `POLLING_NOT_READY`, with the unmet items named in the detail and their keys in `unmet`. Test elections follow the same rule. Fewer than 2 live officials holding Approve raises a warning on the list but does not block. Two identical approval requests cannot both be stored.
- **3.6.10 Member Ballot**: A member on the frozen voter roll fills in every seat, reviews the choices and casts the whole ballot once, with step-up, on web and mobile.
- *Explicit abstain*: each seat needs a candidate or "Abstain on this seat". An untouched seat is undecided and keeps review closed; it is never sent as an abstention. Web and mobile use the same label and help line.
- *Eligibility warning*: the ballot read says whether the member may vote and why not, so the screen can lock the paper early. It authorizes nothing. The vote endpoint checks the roll again and answers 400 to an ineligible member.
- *Ballot order*: the candidate order is set once, when the election enters CandidateList, shuffled or alphabetical as the `CandidateOrder` setting says, and every voter sees it.
- *Receipt*: the vote reply carries a server-made PDF with the tracking code, election title and cast time, and no choices. The client keeps it until the election is Archived or the member signs out. On the web the confirmation screen can also print the marked ballot, from that screen only.
- *Test elections*: an election created as a test run is hidden from the current-election read and from the public results, documents and tracking codes, and declaring it writes no committee members. Admin screens still show it. It needs

the readiness items like any election. It also answers 404 on `GET api/elections/{id}` and `audit/verify`, and it does not hold its EC period, so a rehearsal never blocks the real election. The official PDF route needs sign-in and the `ViewDashboard` election permission. `GET api/elections/latest-declared` is anonymous and returns the newest declared or archived non-test election.

3.7 Alumni Programs & Verification

- **3.7.1 Scholarship & Student-Aid Programme**: Fund → call → application → blind review → award → disbursement workflow for scholarships to schoolchildren. Applicants are identified by a `ReferenceCode` and email, not a Member account. Review queries return a DTO that omits applicant identity, so reviewers cannot obtain it from the API. A paid award writes an idempotent `FinancialRecord` (`Expense/Grant`), linking the disbursement to the ledger.
- **3.7.2 Oral-History / Legacy Archive**: Published collections of recorded alumni testimony, each item carrying a transcript, an uploaded media file, or a link to externally hosted media — at least one of the three is required. The transcript is the searchable, printable product; audio/video is optional.
- **3.7.3 Public Credential Verification**: A publicly reachable, rate-limited endpoint that confirms whether an issued ID card, certificate, or election document is genuine from its QR code or short code, returning only a minimal valid/revoked verdict with no contact or identifying data.

4. Non-Functional Requirements (NFR)

4.1 Security & Data Integrity

- **Archival deletion**: All records use **Soft-Delete (IsArchived)**. Global Query Filters in EF Core ensure archived data never leaks to UI but remains available for audit.
- **Strict Deduplication**: Unique database indexes on NID, Mobile, and Email to prevent record collision.
- **Stateless Auth**: JWT-based session management with real-time **Security Stamp** validation to invalidate sessions if a user is terminated.
- **Rate Limiting**: Tiered limits (Auth: 5/min, Registration: 10/5min, API: 100/min) to slow brute-force attempts and limit abuse.
- **Election records protected**: A user who holds election appointments, approvals or rules unlocks cannot be hard-deleted. The API answers 409 and the user is deactivated instead.
- **Step-up delivery**: The step-up code message names the masked email address. If the code email fails, the user gets a "try again" message (503).

4.2 Performance & UI/UX

- **Server-Side Optimization**: Automatic image compression (< 350KB) for member photos, gallery album photos, and news/event images (payment proofs, signatures, and certificates/documents are intentionally excluded for evidentiary/fidelity reasons); SVG-first document generation.
- **Design system**: one shared theme system with an admin-selectable set of themes, and attention to Largest Contentful Paint on the first page.
- **Resiliency**: Frontend interceptors to handle partial API failures gracefully without crashing user sessions. On mobile, a `RetryInterceptor` retries timeouts, connection errors, and 5xx responses (capped at 3 attempts, exponential backoff) while leaving 4xx/auth failures alone, paired with a connectivity banner shown while offline.

5. System State Logic

Status	Permissions	Directory Visibility
Applied (Partial)	Restricted Onboarding View	Hidden
Applied (Paid)	Restricted "Pending Review"	Hidden
Active	Full Access	Full Visibility (respecting privacy toggles)
Inactive	View Only / Access Denied	Hidden

Status	Permissions	Directory Visibility
Terminated	Account Locked	Hidden
IsArchived	System Hidden	System Hidden (SuperAdmin Audit Only)

6. Infrastructure & Deployment

- **Storage**: Local file storage abstraction with Cloud (S3/Azure) interface readiness.
- **Automation**: Multi-stage **Sequential CI Pipeline** (Analysis -> API Tests -> UI Tests -> Mobile Tests -> Build).
- **Canonical host**: `www.<domain>` is redirected to the main domain read from `AppSettings:ClientUrl` by `WwwRedirectMiddleware`.
- **Observability**: Tiered `ILogger` implementation with SignalR-based real-time admin analytics.
- **Multi-institution deployment (in progress)**: the codebase supports running for an institution other than GHC through an institution profile pack (`profiles/<name>/`), selected by the `ORG_PROFILE` environment variable at deploy time, rather than by editing source. Left unset, behavior is unchanged from today. Not yet safe to use for a second institution until the real alumni data still written into committed migrations is removed.